

Table of Contents

UNIT 4: Control Structures in Python	4
Student Learning Outcomes	4
Introduction	4
4.1 Decision Making	5
4.1.1 The if Statement	5
The Hook — A Story from History	5
The Explanation	5
The Practical Walkthrough — Writing Your First if Statement	6
Flowchart — How the if Statement Works	7
Pause & Think — 4.1.1	7
Quick Recap	7
4.1.2 The if-else Statement	8
The Hook — The Traffic Light	8
The Explanation	8
The Practical Walkthrough — if-else in Action	9
Flowchart — How if-else Works	9
Grab a Partner — 4.1.2	10
Quick Recap	10
4.1.3 Nested Conditions	10
The Hook — The School Gate Guard	10
The Explanation	10
The Practical Walkthrough — Nested if in Action	11
Pause & Think — 4.1.3	12
Quick Recap	12
4.2 Looping Constructs	12
4.2.1 The while Loop	12
The Hook — The Punishment Lines	13
The Explanation	13
The Practical Walkthrough — Counting with a while Loop	14
Pause & Think — 4.2.1	15
Quick Recap	15
4.2.2 The for Loop	15
The Hook — The Teacher Calling the Register	15
The Explanation	15
The Practical Walkthrough — Greeting Each Friend	16

Grab a Partner — 4.2.2	17
Quick Recap	17
4.2.3 The range() Function	17
The Hook — The PE Teacher's Instructions	17
The Explanation	17
The Practical Walkthrough — Using range() in a for Loop	18
Pause & Think — 4.2.3	19
Quick Recap	19
4.2.4 Nested Loops	20
The Hook — The School Timetable	20
The Explanation	20
The Practical Walkthrough — Nested Loops in Action	20
Grab a Partner — 4.2.4	22
Quick Recap	22
4.3 Libraries in Python	22
4.3.1 Using Libraries in Python	22
The Hook — Ada Lovelace and the First Reusable Algorithm	22
The Explanation	23
The Practical Walkthrough — Three Essential Libraries	23
A Useful Shortcut — from ... import	24
Pause & Think — 4.3.1	25
Quick Recap	25
4.4 Lists in Python	25
4.4.1 Creating Lists	25
The Hook — The First Databases	25
The Explanation	26
The Practical Walkthrough — Creating and Printing a List	26
Quick Recap	26
4.4.2 Accessing List Items	27
The Explanation	27
The Practical Walkthrough — Accessing Items by Index	27
Pause & Think — 4.4.2	28
Quick Recap	28
4.4.3 Modifying Lists	28
The Explanation	28
The Practical Walkthrough — Changing and Adding Items	29
Quick Recap	29
4.4.4 Operations on Lists	29
The Hook — The Shopping List	29

The Explanation.....	30
The Practical Walkthrough — All Four Operations	30
Grab a Partner — 4.4.4.....	32
Quick Recap	32
4.5 Testing and Debugging in Python.....	33
4.5.1 Testing	33
The Hook — Re-Reading the Exam Paper	33
The Explanation.....	33
The Practical Walkthrough — Writing Test Cases	33
Pause & Think — 4.5.1	35
Quick Recap	35
4.5.2 Debugging	35
The Hook — Grace Hopper and the Actual Bug	35
The Explanation.....	35
The Practical Walkthrough — Two Worked Debugging Examples.....	36
Pause & Think — 4.5.2	39
Quick Recap	39
Chapter Summary	39

UNIT 4: Control Structures in Python

Student Learning Outcomes

By the end of this chapter, you will be able to:

- Implement control structures such as decision-making statements and loops in Python.
 - Work with Python modules, functions, and built-in data structures like lists.
 - Apply modular programming techniques in Python.
 - Handle exceptions, perform testing, and apply debugging techniques in Python.
-

Introduction

Think about what you did this morning before coming to school.

Maybe you checked the weather. If it was raining, you grabbed an umbrella. If not, you left without one. Then you repeated a task — maybe brushing your teeth, or packing your bag item by item.

You just used two of the most powerful ideas in programming: **making a decision** and **repeating an action**.

In programming, we call these **control structures** — and they are what turn a static list of instructions into a living, thinking program.

Without control structures, a program runs from top to bottom, one line at a time, and stops. That is it. Every line runs exactly once. No decisions. No repetition. No intelligence.

Control structures change that. They let your program:

- **Ask a question** ("Is the temperature above 30?") and choose what to do based on the answer.
- **Repeat an action** ("Print this line 100 times") without you writing it 100 times.

There are two main types of control structures you will learn in this chapter:

Type	What it does	Python tools
Decision Making	Runs different code depending on a condition	<code>if</code> , <code>if-else</code> , nested <code>if</code>
Looping	Repeats code again and again	<code>while</code> , <code>for</code> , <code>range()</code>

You will also learn about **libraries** (toolboxes of pre-built code), **lists** (organized collections of data), and **testing and debugging** (how to check your code and fix it when it breaks).

Let us begin.

4.1 Decision Making

Every single day, you make hundreds of decisions. Should I wear a jacket? Should I study now or later? Should I take the bus or walk?

Your brain checks a **condition** — a question with a yes/no answer — and then chooses what to do.

Python can do exactly the same thing. You give Python a condition, and Python checks it. If the condition is **True**, Python runs one block of code. If it is **False**, Python might run a different block — or skip it entirely.

This is called **decision making**, and it is how your programs become smart.

4.1.1 The **if** Statement

The Hook — A Story from History

In the 1950s, aircraft began using early **autopilot systems**. These systems were not humans. They were machines — and they had to make decisions on their own.

Here is roughly what an early autopilot checked:

"If the altitude is falling below 3,000 feet — activate the climb engine."

Just one condition. One action. But this simple rule, checked hundreds of times per second, kept aeroplanes in the sky.

Today, when you write a Python **if** statement, you are using the exact same logic that engineers wired into those 1950s aircraft — one condition, one outcome.

The Explanation

The **if** statement is the most basic decision in Python.

It says: **"Check this condition. If it is True, run this code."**

Syntax:

```
if condition:
    code to run if condition is True
```

Two things to notice:

1. **The condition** — a question that Python can answer as `True` or `False`. Examples:
`temperature > 30`, `age >= 18`, `name == "Ali"`.
2. **The indented block** — the lines of code that are pushed inward (4 spaces or one Tab). This is called **indentation**. In Python, indentation is **not optional** — it tells Python which lines belong to the `if` statement. If you skip indentation, Python will give you an `IndentationError`.

Plain-language summary: An `if` statement is Python's way of asking a yes/no question. If the answer is yes (True), the indented code runs. If the answer is no (False), Python skips it.

The Practical Walkthrough — Writing Your First `if` Statement

Let us write a program that checks if the temperature is hot.

Step 1: Open your Python editor (IDLE, VS Code, or any tool your teacher uses). Create a new file.

Step 2: Type this code:

```
temperature = 35

if temperature > 30:
    print("It's a hot day!")
```

Step 3: Save and run the program.

Expected Output:

```
It's a hot day!
```

What just happened?

Python looked at `temperature > 30`. Since `temperature` is `35`, this is `35 > 30`, which is `True`. So Python ran the indented line and printed the message.

Step 4: Now change `temperature = 35` to `temperature = 20`. Run again.

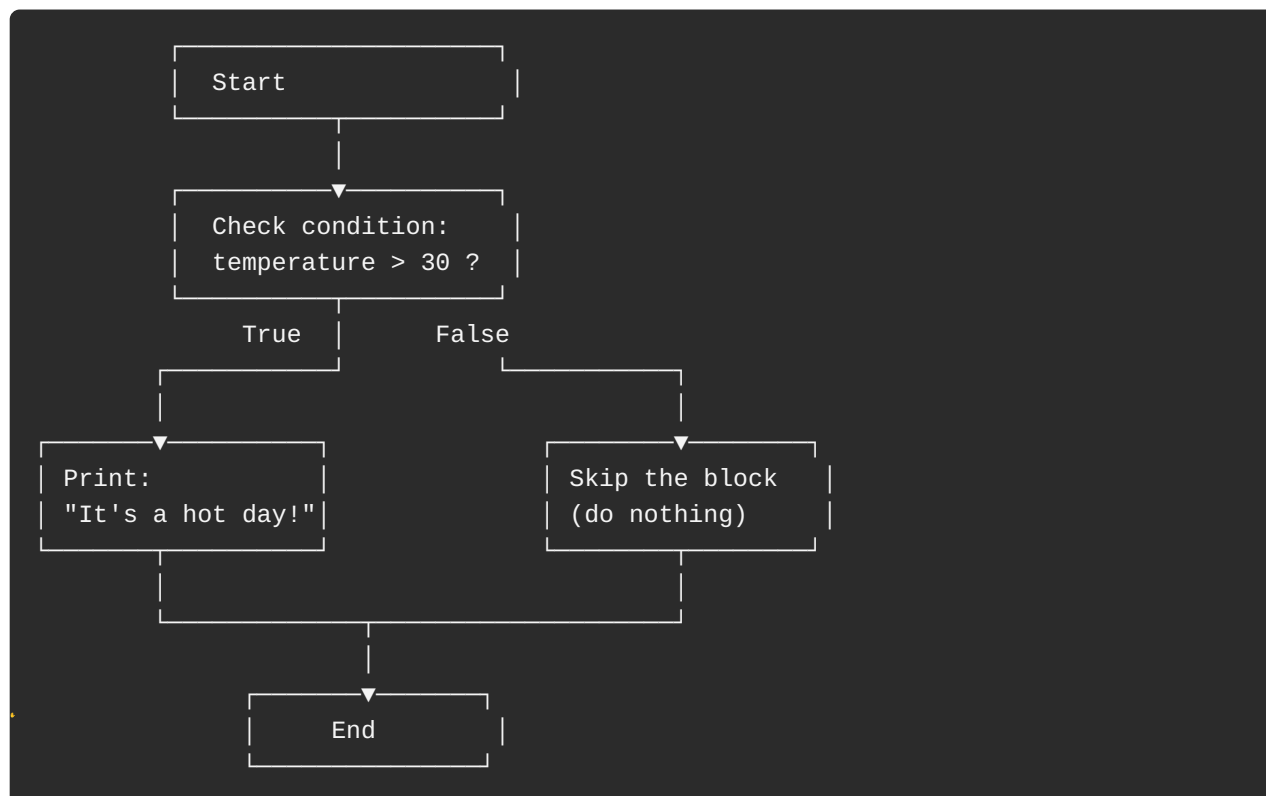
Expected Output:

```
(nothing – the program runs but prints nothing)
```

What just happened?

Now `temperature > 30` is `20 > 30`, which is `False`. The condition failed, so Python **skipped** the indented block entirely.

Flowchart — How the `if` Statement Works



Pause & Think — 4.1.1

A school library only lends books to students who have a valid library card.

Step 1: Write this rule in plain English as an "if" sentence.

(Example: "If the student has a library card, then lend the book.")

Step 2: Now write it in Python. Assume you have a variable called `has_library_card` that is either `True` or `False`.

Hint: Your condition will use a variable, not a number this time.

Quick Recap

- The `if` statement checks a condition. If the condition is **True**, the indented block runs. If it is **False**, Python skips it.

- **Indentation is mandatory** in Python. Always indent the code inside your `if` block.
-

4.1.2 The `if-else` Statement

The Hook — The Traffic Light

Imagine a traffic light at a busy intersection.

It does not simply flash green forever. It makes a decision:

"If the road is clear — show green. Otherwise — show red."

There are exactly **two outcomes**. One condition. Two possibilities. The light must always do one thing or the other — it cannot do nothing.

This is exactly what the `if-else` statement does in Python.

The Explanation

The `if-else` statement handles both sides of a decision.

- If the condition is **True** — run the first block.
- If the condition is **False** — run the second block (`else`).

Syntax:

```
if condition:
    code to run if condition is True
else:
    code to run if condition is False
```

Plain-language summary: `if-else` is like a fork in the road. Your program will always take one path or the other — it will never stand still.

The Practical Walkthrough — `if-else` in Action

```
temperature = 15

if temperature > 30:
    print("It's a hot day!")
else:
    print("It's not a hot day.")
```

Expected Output:

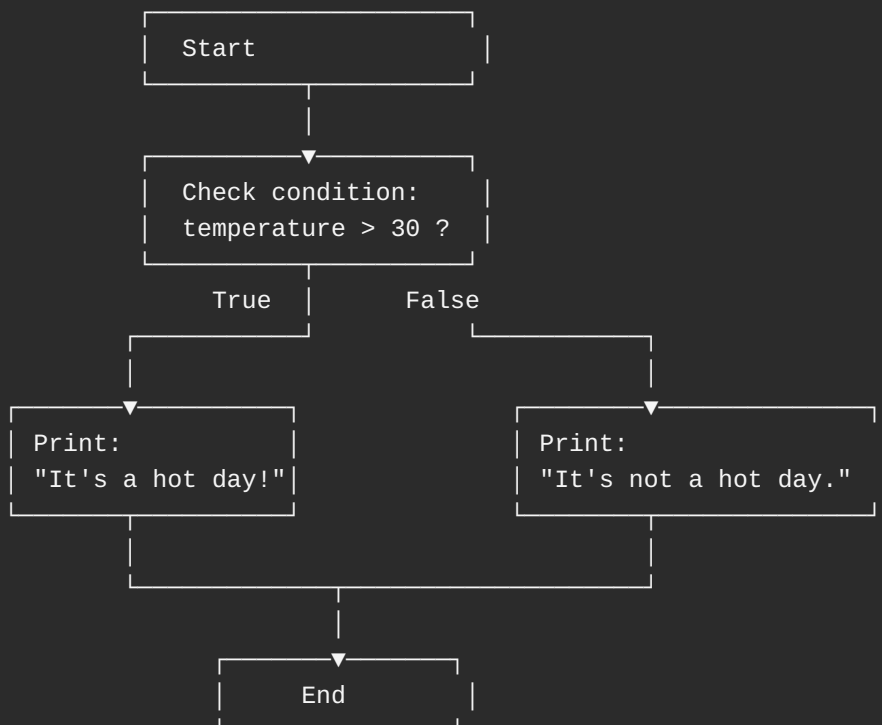
```
It's not a hot day.
```

What just happened?

`temperature` is `15`. So `15 > 30` is `False`. Python skipped the `if` block and ran the `else` block instead.

Now try it yourself: Change `temperature` to `40`. What output do you expect? Run it and check.

Flowchart — How `if-else` Works



Grab a Partner — 4.1.2

One partner: Think of a real-life decision that has exactly **two outcomes**. Write it in plain English.

(Examples: "If I have money, I buy a samosa. Otherwise I skip the canteen." or "If the alarm rings, I wake up. Otherwise I keep sleeping.")

Other partner: Take that sentence and write it as a Python `if-else` block. Use any variable name and value that fits.

Swap roles and repeat with a new scenario.

Bonus question: Can you think of a real-life decision that has MORE than two outcomes? (We will learn how to handle that with nested conditions next!)

Quick Recap

- `if-else` gives your program **two paths**: one for `True`, one for `False`.
- Your program will **always** take one of the two paths — never both, never neither.

4.1.3 Nested Conditions

The Hook — The School Gate Guard

Imagine a school gate guard with two jobs:

1. First, check if the student has their **ID card**.
2. If yes — then check if they are wearing the **proper uniform**.

Only if **both** checks pass does the student get in.

Notice the structure: one check happens **inside** another check. The second question only gets asked if the first one is already answered "yes."

This is **nesting** — and in Python, it means putting one `if` statement **inside** another.

The Explanation

A **nested condition** is simply an `if` (or `if-else`) statement placed **inside** another `if` block.

Python checks the **outer condition** first. Only if it is `True` does Python move inside and check the **inner condition**.

Syntax:

```
if condition1:
    if condition2:
        code to run if BOTH condition1 AND condition2 are True
    else:
        code to run if condition1 is True but condition2 is False
else:
    code to run if condition1 is False
```

Plain-language summary: Nested conditions are checks inside checks. Python only reads the inner check if the outer check is already True.

The Practical Walkthrough — Nested `if` in Action

Let us code the weather scenario from the textbook:

```
weather = "rainy"
temperature = 10

if weather == "rainy":
    if temperature < 15:
        print("Wear a raincoat!")
    else:
        print("Take an umbrella.")
else:
    print("Enjoy your day!")
```

Expected Output:

```
Wear a raincoat!
```

Let us trace through this step by step:

Step	What Python checks	Result
1	<code>weather == "rainy"</code> → is <code>"rainy" == "rainy"</code> ?	True → enter the outer <code>if</code> block
2	<code>temperature < 15</code> → is <code>10 < 15</code> ?	True → print <code>"Wear a raincoat!"</code>

Now experiment:

- Change `temperature = 10` to `temperature = 20`. What happens? (Outer condition is still True, but inner condition is now False → "Take an umbrella.")

- Change `weather = "rainy"` to `weather = "sunny"`. What happens now? (Outer condition is False → skip everything → "Enjoy your day!")
-

Pause & Think — 4.1.3

You want to grant access to the school computer lab. But there are **two rules**:

- Rule 1: The student must have a **lab pass**.
- Rule 2: It must **not be a holiday**.

Step 1: Write this logic in plain English using "if" sentences.

Step 2: Write it in Python. Use variables `has_lab_pass = True` and `is_holiday = False`. Try changing their values and predicting the output before you run.

Quick Recap

- Nested conditions put one `if` statement **inside** another.
 - The **inner check only runs if the outer check is True**.
 - Use nested conditions when a second question only makes sense after the first one is answered.
-

4.2 Looping Constructs

Imagine you are a teacher. You need to call out every student's name from a register of 40 students. You say each name, wait for the student to respond, then move to the next.

Would you write 40 separate `print()` statements? Of course not.

That is what **loops** are for. A loop lets you repeat an action — once, ten times, a million times — without copying and pasting the same code over and over.

Loops are one of the most powerful tools in any programming language. Let us learn both kinds Python gives us.

4.2.1 The `while` Loop

The Hook — The Punishment Lines

You are caught talking in class. Your teacher says:

"Write 'I will not talk in class' fifty times."

So you pick up your pen. You write the line. You check your count. Is it 50 yet? No. You write again. You check again. Still not 50. You keep going — until finally, after the 50th line, the condition is met and you stop.

This is **exactly** how a `while` loop works.

The Explanation

A `while` loop keeps running a block of code **as long as a condition is True**. It checks the condition before every single repetition. The moment the condition becomes **False**, the loop stops.

Syntax:

```
while condition:
    code to run while condition is True
```

Plain-language summary: A `while` loop asks "is this still True?" before every round. As soon as the answer is "no," it stops.

Three key ingredients every `while` loop needs:

1. A **starting value** (e.g., `number = 1`)
2. A **condition** to check (e.g., `number < 10`)
3. An **update** that changes the value so the condition eventually becomes False (e.g., `number = number + 1`)

Important warning — The Infinite Loop: If you forget ingredient 3 (the update), your condition will **never** become False. The loop will run forever. This is called an **infinite loop**. Every programmer accidentally writes one at some point. Do not panic — just press **Ctrl + C** to stop the program, then fix your code. It is not a disaster. It is a learning moment.

The Practical Walkthrough — Counting with a `while` Loop

```
number = 1

while number < 10:
    print(number)
    number = number + 1
```

Expected Output:

```
1
2
3
4
5
6
7
8
9
```

Trace Table — what happens at each step:

Iteration	Value of <code>number</code> before check	<code>number < 10</code> ?	Action	Value of <code>number</code> after update
1	1	True	Print <code>1</code>	2
2	2	True	Print <code>2</code>	3
3	3	True	Print <code>3</code>	4
4	4	True	Print <code>4</code>	5
5	5	True	Print <code>5</code>	6
6	6	True	Print <code>6</code>	7
7	7	True	Print <code>7</code>	8
8	8	True	Print <code>8</code>	9
9	9	True	Print <code>9</code>	10
10	10	False	Stop — loop ends	—

Notice that `10` is **never printed**, because the condition `number < 10` becomes False when `number` reaches `10`, and the loop stops before printing.

Pause & Think — 4.2.1

A water tank is empty. It fills at **10 litres per minute**. The tank is full at **100 litres**.

Write a **while** loop that:

- Starts at 0 litres
- Adds 10 litres each minute
- Prints the current water level after each minute
- Stops when the tank is full (100 litres)

Before you code: Draw your trace table on paper first. What will the output look like?

Quick Recap

- A **while** loop runs **as long as its condition is True** and stops the moment it becomes False.
- Always include an **update** inside the loop, or it will run forever.

4.2.2 The **for** Loop

The Hook — The Teacher Calling the Register

Every morning, your teacher opens the class register. It has 40 names. The teacher calls name number 1, waits, marks attendance. Then name number 2. Then number 3. All the way to the end.

The teacher does not decide when to stop — the register decides. When the list is finished, the teacher stops.

This is the **for** loop. It goes through a sequence — a list, a range of numbers, or any collection of items — one item at a time, from beginning to end.

The Explanation

A **for** loop repeats code for each item in a sequence. It automatically moves from one item to the next and stops when it reaches the end.

Syntax:

```
for variable in sequence:
    code to run for each item
```

- **variable** — a temporary name that holds the **current item** during each repetition. You choose the name.
- **sequence** — a collection of items (like a list or a range of numbers).

Plain-language summary: A **for** loop visits every item in a sequence, one by one, and runs your code for each one. When there are no more items, it stops.

The Practical Walkthrough — Greeting Each Friend

```
friends = ["Sami", "Raza", "Moosa"]

for friend in friends:
    print("Welcome:", friend)
```

Expected Output:

```
Welcome: Sami
Welcome: Raza
Welcome: Moosa
```

Trace Table — what happens each round:

Round	Value of friend	What prints
1	"Sami"	Welcome: Sami
2	"Raza"	Welcome: Raza
3	"Moosa"	Welcome: Moosa
—	(list is empty)	Loop ends

Notice: you wrote the **print()** line **once**, but it ran **three times** — once for each friend.

Grab a Partner — 4.2.2

One partner: On a piece of paper, write a list of **five Pakistani cities**.

Other partner: Write a `for` loop in Python that prints each city along with its position number (1, 2, 3, 4, 5).

(Hint: you may need to create a variable outside the loop to keep track of the position number. Can you figure out how?)

Discuss together: What would change in your code if the list had 10 cities instead of 5? Would you need to change the loop at all?

Quick Recap

- A `for` loop visits **each item in a sequence**, one by one, and runs your code for each.
- It **automatically stops** when all items have been visited — no condition needed.

4.2.3 The `range()` Function

The Hook — The PE Teacher's Instructions

Your PE teacher blows the whistle and says:

"Do 10 jumping jacks. Start from 1. Count up by 1 each time. Stop at 10."

Three pieces of information: where to start, where to stop, and how many to count per step.

Python's `range()` function works exactly like this. You give it a start, a stop, and a step — and it generates a sequence of numbers for you.

The Explanation

The `range()` function generates a sequence of numbers. It is almost always used with a `for` loop to repeat code a specific number of times.

Three ways to use `range()`:

Syntax	What it does	Example output
<code>range(stop)</code>	Counts from 0 to <code>stop - 1</code>	<code>range(5)</code> → 0, 1, 2, 3, 4
<code>range(start, stop)</code>	Counts from <code>start</code> to <code>stop - 1</code>	<code>range(1, 6)</code> → 1, 2, 3, 4, 5
<code>range(start, stop, step)</code>	Counts from <code>start</code> to <code>stop - 1</code> , jumping by <code>step</code>	<code>range(0, 10, 2)</code> → 0, 2, 4, 6, 8

Important: `range()` always stops **before** the stop number. So `range(5)` gives you 0, 1, 2, 3, 4 — **not** 5. This trips up almost every beginner. You are not alone.

The Practical Walkthrough — Using `range()` in a `for` Loop

Example 1: Print 0 to 4

```
for i in range(5):
    print(i)
```

Expected Output:

```
0
1
2
3
4
```

Example 2: Print 1 to 5

```
for i in range(1, 6):
    print(i)
```

Expected Output:

```
1
2
3
4
5
```

Example 3: Print even numbers from 2 to 10, on one line

```
for i in range(2, 11, 2):
    print(i, end=" ")
```

Expected Output:

```
2 4 6 8 10
```

What is `end=" "`? By default, `print()` moves to a new line after each output. Adding `end=" "` tells `print()` to put a **space** instead of a new line. This keeps everything on one line.

Trace Table — for `range(2, 11, 2)`:

Round	Value of <code>i</code>	<code>i < 11</code> ?	Output
1	2	True	<code>2</code>
2	4	True	<code>4</code>
3	6	True	<code>6</code>
4	8	True	<code>8</code>
5	10	True	<code>10</code>
6	12	False	Loop ends

Pause & Think — 4.2.3

Look at this: `range(1, 20, 3)`

Before you run it: On paper, write down every number you think this will produce. Explain your reasoning.

Then: Write a `for` loop that uses `range(1, 20, 3)` and prints each number. Run the code.

Were you right? If not — which part of the `range()` rule did you misunderstand? (That is the part to remember!)

Quick Recap

- `range()` generates a sequence of numbers — useful for repeating something a set number of times.
- It always **stops before** the stop number.
- Use `range(start, stop, step)` when you need more control over the sequence.

4.2.4 Nested Loops

The Hook — The School Timetable

Picture your school timetable. For every **day of the week** (Monday to Friday), there are **6 periods** per day.

If you want to list every single period of every single day, you go through it like this:

- Monday: Period 1, Period 2, Period 3, Period 4, Period 5, Period 6
- Tuesday: Period 1, Period 2, ... Period 6
- Wednesday: Period 1, Period 2, ... Period 6
- ...

An **outer loop** runs through the days. For **each day**, an **inner loop** runs through all the periods. The inner loop completes its full cycle **every single time** the outer loop takes one step.

This is a **nested loop** — a loop inside a loop.

The Explanation

A **nested loop** is simply a loop placed **inside another loop**.

- The **outer loop** controls the big repetition (e.g., days of the week).
- The **inner loop** controls the small repetition (e.g., periods in each day).
- For **every single iteration** of the outer loop, the **inner loop runs completely from start to finish**.

Plain-language summary: In nested loops, the inner loop finishes its entire job before the outer loop takes its next step.

The Practical Walkthrough — Nested Loops in Action

Example 1: A Number Grid Pattern

```
print("Basic Number Pattern")

for i in range(3):           # Outer loop – runs 3 times
    for j in range(4):       # Inner loop – runs 4 times for EACH outer loop step
        print(f"({i},{j})", end=" ")
    print()                 # Move to next line after inner loop finishes
```

Expected Output:

```
Basic Number Pattern
(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
```

Example 2: A Right-Angled Triangle of Stars

```
n = 5

for i in range(1, n + 1):      # Outer loop: i goes from 1 to 5
    for j in range(i):        # Inner loop: runs 'i' times
        print("*", end=" ")
    print()                   # New line after each row
```

Expected Output:

```
*
* *
* * *
* * * *
* * * * *
```

Trace Table — for the triangle (outer loop only):

Outer loop value (i)	Inner loop runs how many times?	Row printed
1	1	
2	2	
3	3	
4	4	
5	5	

Grab a Partner — 4.2.4

Before writing any code:

On paper, **draw the output** of this nested loop:

```
python
for i in range(3):
    for j in range(3):
        print("*", end=" ")
    print()
```

Draw exactly what you think will appear on the screen — a grid of asterisks.

Then: Type the code and run it. Does your drawing match?

Discuss: How would you change the code to make the grid 5×5 instead of 3×3?

Quick Recap

- A nested loop is a **loop inside a loop**.
- The **inner loop runs completely** every time the outer loop takes one step.
- Use nested loops for patterns, grids, and any task that has "for each X, do Y repeatedly."

4.3 Libraries in Python

4.3.1 Using Libraries in Python

The Hook — Ada Lovelace and the First Reusable Algorithm

In 1843, a mathematician named **Ada Lovelace** did something no one had done before. She was writing instructions for a mechanical computing machine, and she noticed a problem: some calculations needed to be repeated over and over in different parts of the program.

Instead of writing the same calculation ten times, she wrote it **once** — in a reusable block of code that could be called from anywhere. This idea — write something once, use it anywhere — is the foundation of what we now call a **library**.

Over 150 years later, when you type `import math` in Python, you are using Ada Lovelace's idea. Someone wrote that code once, carefully, and made it available to every programmer in the world.

The Explanation

A **library** (also called a **module**) is a collection of ready-made code that someone else has already written and tested. Instead of building every tool from scratch, you **import** a library and use its tools.

Think of it like a **toolbox**. You do not build a screwdriver from raw metal every time you need to tighten a screw. You reach into your toolbox and pick it up.

How to import a library:

```
import library_name
```

How to use a function from a library:

```
library_name.function_name(arguments)
```

The Practical Walkthrough — Three Essential Libraries

Library 1: **math** — for mathematical calculations

```
import math

# Calculate square root
result = math.sqrt(25)
print("Square root of 25:", result)

# Use the value of pi
print("Value of pi:", math.pi)

# Calculate power
print("2 to the power of 8:", math.pow(2, 8))
```

Expected Output:

```
Square root of 25: 5.0
Value of pi: 3.141592653589793
2 to the power of 8: 256.0
```

Library 2: **random** — for generating random numbers

```
import random

# Generate a random whole number between 1 and 10
number = random.randint(1, 10)
print("The random number is:", number)
```

Expected Output (will vary each time — that is the point!):

```
The random number is: 7
```

Library 3: `statistics` — for analysing data

```
import statistics

data = [23, 45, 67, 89, 12, 44, 56]

mean_value = statistics.mean(data)
print("The mean value is:", mean_value)
```

Expected Output:

```
The mean value is: 48
```

A Useful Shortcut — `from ... import ...`

Sometimes you only need one specific tool from a library. You can import just that tool:

```
from math import sqrt, pi

print(sqrt(49))
print(pi)
```

Expected Output:

```
7.0
3.141592653589793
```

This way, you can write `sqrt()` instead of `math.sqrt()` — it is shorter and faster when you only need a few tools.

Pause & Think — 4.3.1

You need to calculate the area of a circle. The formula is:

$$\text{Area} = \pi \times \text{radius}^2$$

Without the `math` library: What would you have to type every time you use the value of pi? (Think about how many decimal places you would need to remember!)

With `import math`: Write the Python code that calculates the area of a circle with radius = 7. Use `math.pi` and `math.pow()`.

Bonus: What other calculations might become easier with the `math` library? Can you name two?

Quick Recap

- A **library** is a collection of ready-made code you can use in your own programs.
- Use `import library_name` to load a library.
- Use `library_name.function_name()` to call a function from it.

4.4 Lists in Python

4.4.1 Creating Lists

The Hook — The First Databases

In the early days of computing — the 1950s and 1960s — businesses needed to store customer records. Engineers solved this by setting aside a **sequence of slots in memory**, one after another, each holding one piece of data.

Customer #1 in slot 1. Customer #2 in slot 2. And so on.

You needed to know the **position number** of a record to find it instantly. It was fast, ordered, and expandable.

Today, Python's **list** is the direct descendant of that idea. A list is an ordered collection of items, stored in numbered positions. You can add to it, remove from it, change it, and search through it — all with simple commands.

The Explanation

A **list** is a variable that holds **multiple values** in a specific order.

- Lists are created using **square brackets** `[]`.
- Items inside the list are separated by **commas**.
- A list can hold **any type of data** — numbers, text, or a mix.

Syntax:

```
list_name = [item1, item2, item3]
```

The Practical Walkthrough — Creating and Printing a List

```
fruits = ["Mango", "Apple", "Banana"]  
print(fruits)
```

Expected Output:

```
['Mango', 'Apple', 'Banana']
```

An empty list (a list with nothing in it yet):

```
empty_list = []  
print(empty_list)
```

Expected Output:

```
[]
```

A list with mixed types:

```
mixed = ["Ali", 17, True, 3.14]  
print(mixed)
```

Expected Output:

```
['Ali', 17, True, 3.14]
```

Quick Recap

- A list holds **multiple values** in one variable, in order.
- Create a list with square brackets: `my_list = [item1, item2, item3]`.

4.4.2 Accessing List Items

The Explanation

Every item in a list has an **index** — a number that tells you its position.

Critical fact: Python counts from **0**, **not 1**. The first item is at index **0**, the second at index **1**, and so on. This surprises almost every beginner. You are not alone — even professional programmers double-check this sometimes.

Index diagram:

```
fruits = ["Mango", "Apple", "Banana", "Guava"]
          ↑       ↑       ↑       ↑
          [0]     [1]     [2]     [3]
```

Negative indexes count from the end:

```
fruits = ["Mango", "Apple", "Banana", "Guava"]
          ↑       ↑       ↑       ↑
          [-4]    [-3]    [-2]    [-1]
```

- `fruits[-1]` gives the **last** item: `"Guava"`
- `fruits[-2]` gives the **second to last**: `"Banana"`

The Practical Walkthrough — Accessing Items by Index

```
fruits = ["Mango", "Apple", "Banana", "Guava"]

print(fruits[0])    # First item
print(fruits[1])    # Second item
print(fruits[-1])   # Last item
print(fruits[-2])   # Second to last item
```

Expected Output:

```
Mango
Apple
Guava
Banana
```

What happens if you use an index that does not exist?

```
print(fruits[10])
```

Error Output:

```
IndexError: list index out of range
```

Python is telling you: "I looked for position 10, but this list only has 4 items (positions 0 to 3). Position 10 does not exist."

Remember: An `IndexError` is Python helping you. It is not shouting — it is telling you exactly what went wrong and where.

Pause & Think — 4.4.2

Given this list:

```
python
fruits = ["Mango", "Apple", "Banana", "Guava"]
```

Predict first — then test:

- What is `fruits[0]`?
- What is `fruits[-1]`?
- What would happen if you tried `fruits[10]`?

Write your predictions on paper, **then** run the code. Were you right? What did the error message tell you?

Quick Recap

- List indexes start at **0**, not 1. The first item is `list[0]`.
- Use **negative indexes** to count from the end. `list[-1]` is always the last item.
- Trying an index that does not exist gives an `IndexError`.

4.4.3 Modifying Lists

The Explanation

Lists in Python are **mutable** — that means you can **change** them after they are created.

You can:

- **Change** an existing item by assigning a new value to its index.
 - **Add** a new item to the end using `.append()`.
-

The Practical Walkthrough — Changing and Adding Items

```
fruits = ["Mango", "Apple", "Banana"]

# Change the first item
fruits[0] = "Orange"
print(fruits)

# Add a new item to the end
fruits.append("Pineapple")
print(fruits)
```

Expected Output:

```
['Orange', 'Apple', 'Banana']
['Orange', 'Apple', 'Banana', 'Pineapple']
```

What just happened?

1. `fruits[0] = "Orange"` replaced `"Mango"` (at position 0) with `"Orange"`.
 2. `.append("Pineapple")` added `"Pineapple"` to the **end** of the list.
-

Quick Recap

- Change an item: `list[index] = new_value`.
 - Add an item to the end: `list.append(new_value)`.
-

4.4.4 Operations on Lists

The Hook — The Shopping List

Think of a paper shopping list. You can:

- **Add** a new item when you remember something.
- **Remove** an item when you bought it.
- **Sort** the list alphabetically to make shopping easier.
- **Flip** the list order if you want to start from the bottom of the shop.

Python lists work exactly the same way — with built-in methods for each of these operations.

The Explanation

Python gives you built-in **methods** (actions) to work with lists:

Method	What it does
<code>.append(item)</code>	Adds an item to the end of the list
<code>.remove(item)</code>	Removes the first occurrence of the item
<code>.sort()</code>	Sorts the list in ascending order (A to Z, smallest to largest)
<code>.reverse()</code>	Reverses the order of the list
<code>len(list)</code>	Returns the number of items in the list

The Practical Walkthrough — All Four Operations

Example 1: `.append()` — Adding a student

```
students = ["Ahmed", "Sara", "Ali"]
students.append("Hina")
print(students)
```

Expected Output:

```
['Ahmed', 'Sara', 'Ali', 'Hina']
```

Example 2: `.remove()` — Removing a student

```
students = ["Ahmed", "Sara", "Ali", "Hina", "Sara"]
students.remove("Sara")
print(students)
```

Expected Output:

```
['Ahmed', 'Ali', 'Hina', 'Sara']
```

Note: `.remove()` only removes the **first** occurrence. Notice that the second `"Sara"` is still in the list!

Example 3: `.sort()` — Sorting numbers

```
numbers = [34, 12, 5, 89, 23]
numbers.sort()
print(numbers)
```

Expected Output:

```
[5, 12, 23, 34, 89]
```

Example 4: `.reverse()` — Reversing a list

```
fruits = ["Apple", "Banana", "Orange", "Mango"]
fruits.reverse()
print(fruits)
```

Expected Output:

```
['Mango', 'Orange', 'Banana', 'Apple']
```

Example 5: `len()` — Counting items

```
fruits = ["Apple", "Banana", "Mango"]
print("Number of fruits:", len(fruits))
```

Expected Output:

```
Number of fruits: 3
```

Putting it all together — a complete example:

```
# Start with a class list
students = ["Ahmed", "Sara", "Ali"]

# Add a new student
students.append("Hina")

# One student left
students.remove("Sara")

# Sort alphabetically
students.sort()

print("Final class list:", students)
print("Number of students:", len(students))
```

Expected Output:

```
Final class list: ['Ahmed', 'Ali', 'Hina']
Number of students: 3
```

Grab a Partner — 4.4.4

One partner: Write a list of **five numbers** on paper — any five numbers you like.

Other partner: Using only the list operations from this section, write Python code that finds:

- The **largest** number (Hint: sort the list first, then access the last item)
- The **sum** of all numbers (Hint: Python has a built-in `sum()` function!)
- The **length** of the list (how many numbers)

Swap roles with a different set of five numbers.

Quick Recap

- Python lists have powerful built-in methods:
`.append()`, `.remove()`, `.sort()`, `.reverse()`.
 - `len()` tells you how many items are in a list.
 - `.remove()` only removes the **first matching item**, not all of them.
-

4.5 Testing and Debugging in Python

4.5.1 Testing

The Hook — Re-Reading the Exam Paper

You finish your exam with 10 minutes to spare. A good student does not sit back and relax. They go back through every answer — checking, verifying, trying to catch mistakes before the teacher does.

This is **testing**.

In programming, testing means **running your code with different inputs to make sure it behaves correctly in all situations** — not just the obvious ones.

The Explanation

Testing is the process of checking that your program does what it is supposed to do.

Here is an important truth: **no program is correct just because it ran without errors**. A program can run perfectly and still give the wrong answer. Testing is how you catch these problems before they cause real harm.

Types of Testing:

Type	What it checks	Example
Unit Testing	One small part of the code in isolation (e.g., one function)	Test that <code>add(2, 3)</code> returns <code>5</code>
Integration Testing	How different parts of the code work together	Test that the login system works with the database
Functional Testing	Whether the software does what the user expects	Test that clicking "Submit" actually saves the form
Regression Testing	That fixing one bug did not accidentally break something else	After fixing the login page, check the signup page still works

The Practical Walkthrough — Writing Test Cases

A **test case** is one specific input + the expected output you would check against.

Imagine you write this function to add two numbers:

```
def add(a, b):
    return a + b
```

Good test cases to write:

Test case	Input	Expected output	Why this test?
Normal case	<code>add(2, 3)</code>	<code>5</code>	Basic functionality
Zero	<code>add(0, 5)</code>	<code>5</code>	Edge case — does zero cause problems?
Negative numbers	<code>add(-3, 7)</code>	<code>4</code>	Are negatives handled?
Large numbers	<code>add(999999, 1)</code>	<code>1000000</code>	Does it work at scale?

Using Python's `unittest` module:

```
import unittest

def add(a, b):
    return a + b

class TestAdd(unittest.TestCase):
    def test_normal(self):
        self.assertEqual(add(2, 3), 5)

    def test_zero(self):
        self.assertEqual(add(0, 5), 5)

    def test_negative(self):
        self.assertEqual(add(-3, 7), 4)

if __name__ == "__main__":
    unittest.main()
```

Expected Output (if all tests pass):

```
...
-----
Ran 3 tests in 0.001s

OK
```

Each `.` represents one passed test. `OK` means all tests passed.

Pause & Think — 4.5.1

You write a program to calculate the **average** of three numbers. The formula is:

```
average = (number1 + number2 + number3) / 3
```

Your task: Think of **at least three test cases** you would use to check this program. For each one, write:

- The three input numbers
- The expected average output

Challenge: Can you think of an input that might cause a problem or produce an unexpected result? (Hint: what if all three numbers are 0?)

Quick Recap

- **Testing** is running your code with various inputs to make sure it works correctly.
- Always test **normal cases**, **edge cases** (unusual values like 0 or very large numbers), and **cases that might break the program**.

4.5.2 Debugging

The Hook — Grace Hopper and the Actual Bug

On **September 9th, 1947**, engineers at Harvard University were working on a large computer called the **Mark II**. The machine kept producing errors. After hours of searching, they found the problem: a **real moth** had flown inside the computer and gotten stuck in one of the electrical relays, stopping it from working.

The lead engineer, **Grace Hopper**, carefully removed the moth and taped it into the logbook with a note: *"First actual case of bug being found."*

That is where programmers get the word **"bug"** — and why fixing a bug is called **"debugging."**

Today, your bugs are not moths. They are typos, wrong conditions, and logical mistakes. But Grace Hopper's approach is still exactly right: **find the bug, remove it, and write down what you learned.**

The Explanation

Debugging is the process of finding and fixing errors in your code.

There are two main types of errors in Python:

Error Type	What it means	Example
Syntax Error	Python cannot understand your code — it is grammatically wrong	Missing colon, wrong spelling of a keyword
Logic Error	Python runs the code without crashing, but the output is wrong	Using <code>+</code> instead of <code>-</code> , wrong condition

The most important mindset shift: An error message is **not Python insulting you**. It is Python pointing a finger at the line where it got confused and asking for help. A programmer who can read an error message and act on it is already thinking like a professional.

Three debugging techniques:

1. **Print Statements** — Add `print()` lines inside your code to see the value of variables at different stages.
2. **Read the Error Message** — Python tells you the **line number** and the **type of error**. Start there.
3. **Trace Through Manually** — Go through your code line by line on paper, pretending to be Python. What is the value of each variable at each step?

The Practical Walkthrough — Two Worked Debugging Examples

Debugging Example 1: Syntax Error

The broken code:

```
temperature = 35

if temperature > 30
    print("It's a hot day!")
```

The error message Python gives:

```
File "example.py", line 3
    if temperature > 30
        ^
SyntaxError: expected ':'
```

Diagnosis:

- Python says the error is on **line 3**.
- The error type is **SyntaxError**.
- The message says **expected ':'** — Python expected a colon at the end of the **if** statement, but did not find one.

The fix:

```
temperature = 35

if temperature > 30:          # ← Added the missing colon here
    print("It's a hot day!")
```

Corrected Output:

```
It's a hot day!
```

What we learned: Every **if**, **while**, **for**, and **else** line in Python **must end with a colon** **:**. Forgetting it is one of the most common beginner mistakes — and one of the easiest to fix.

Debugging Example 2: Logic Error

The task: Write a program that counts down from 10 to 1 and prints "Lift off!" at the end.

The broken code:

```
number = 10

while number > 0:
    print(number)
    number = number + 1      # ← Bug is here

print("Lift off!")
```

What happens when you run this?

The output starts:

```
10
11
12
13
...
```

And keeps going forever. **This is an infinite loop.**

There is **no error message** — Python is not confused. It is doing exactly what you told it to do. The bug is in your **logic**, not your grammar.

Diagnosis:

- The loop should be counting **down** (towards 0), but `number = number + 1` is making it count **up** (away from 0).
- The condition `number > 0` will never become False because `number` is growing, not shrinking.

The fix:

```
number = 10

while number > 0:
    print(number)
    number = number - 1    # ← Changed + to -

print("Lift off!")
```

Corrected Output:

```
10
9
8
7
6
5
4
3
2
1
Lift off!
```

What we learned: Logic errors do not produce error messages — they produce **wrong output** (or in this case, infinite output). Always trace through your loop logic manually on paper first: "Is my variable moving towards making the condition False, or away from it?"

Pause & Think — 4.5.2

The code below is supposed to print the sum of all numbers from 1 to 5. But something is wrong.

```
```python
total = 0

for i in range(1, 5):
 total = total + i

print("Sum:", total)
```
```

The output is:

Sum: 10

But the correct answer should be 15 ($1 + 2 + 3 + 4 + 5 = 15$).

- Can you find the bug?
- Is it a syntax error or a logic error?
- What is the fix?
- Write down what you learned from this bug.

Quick Recap

- **Debugging** is finding and fixing errors. There are two main types: **syntax errors** (bad grammar — Python cannot run the code) and **logic errors** (bad thinking — Python runs but gives wrong output).
- **Read error messages carefully** — they tell you the line number and the type of problem.
- **Logic errors are sneakier** — use print statements and trace tables to hunt them down.

Chapter Summary

Congratulations. You have just covered the most important building blocks of programming logic.

| What you learned | The key idea |
|--------------------------------|---|
| <code>if</code> statement | Runs code only if a condition is True |
| <code>if-else</code> statement | Chooses between two paths based on a condition |
| Nested conditions | Checks inside checks — one condition inside another |
| <code>while</code> loop | Repeats as long as a condition is True |
| <code>for</code> loop | Repeats for each item in a sequence |
| <code>range()</code> function | Generates a sequence of numbers for use in loops |
| Nested loops | A loop inside a loop — inner loop runs fully each outer step |
| Libraries | Ready-made toolboxes of code you can import and use |
| Lists | Ordered collections of items, changeable, indexed from 0 |
| Testing | Checking your code with various inputs to verify it works |
| Debugging | Finding and fixing errors — syntax errors and logic errors |

One final thought from your instructor:

Every concept in this chapter — every `if`, every loop, every list — was once new and confusing to a professional programmer. The engineers at NASA, the developers at Google, the security researchers at top firms — they all learned these exact same ideas in their first programming course.

What made them professionals was not talent. It was persistence. They ran the code. They read the error message. They fixed it. They ran it again.

That is the entire process. You already started it.

End of Chapter 4: Control Structures in Python